

Processes, Scheduling and Memory

Course notes - Department of Computer Science

1. Processes and threads

A process is a program in execution together with the state the kernel maintains for it: address space, open file descriptors, and scheduling information. Threads share an address space within a process, which makes communication cheap and synchronisation mandatory.

Context switching between processes requires swapping address spaces and flushing translation caches, which is why thread switches are markedly cheaper.

2. CPU scheduling

First-come first-served is trivial but suffers convoy effects, where a long job delays every short one behind it. Shortest-job-first minimises average waiting time but needs knowledge of burst lengths and can starve long jobs.

Round-robin assigns each process a time quantum and cycles through the ready queue, trading throughput for responsiveness. Multilevel feedback queues approximate shortest-job-first adaptively by demoting processes that use their full quantum.

3. Synchronisation

A race condition arises when the result depends on the interleaving of concurrent accesses to shared state. The critical section problem asks for mutual exclusion, progress and bounded waiting.

Mutexes provide exclusion, semaphores generalise to counted resources, and condition variables let a thread wait for a predicate to become true. Monitors bundle the lock with the data it protects, which is the pattern most modern languages expose.

4. Deadlock

Deadlock requires four simultaneous conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. Breaking any one of them prevents deadlock.

Systems either prevent deadlock by design - most often by imposing a global lock ordering, which breaks circular wait - or detect it and recover. Avoidance algorithms such as the banker's algorithm exist but need advance knowledge of maximum resource claims and are rarely practical.

5. Virtual memory

Paging maps fixed-size virtual pages onto physical frames through a page table, giving each process an isolated address space larger than physical memory. The translation lookaside buffer caches recent translations, without which every access would cost an extra memory reference.

When a referenced page is not resident, a page fault traps to the kernel, which loads the page and possibly evicts another. Replacement policies approximate the optimal algorithm; least-recently-used and clock are the usual choices. Thrashing occurs when the working set exceeds available frames and the system spends its time

paging rather than computing.